Homework 5 - Deep Learning

Mirjeta Misini

03 12 2019

Packages :

```
library(keras)
library(caret)
```

```
## Loading required package: lattice
```

```
## Loading required package: ggplot2
```

```
library(ggfortify)
```

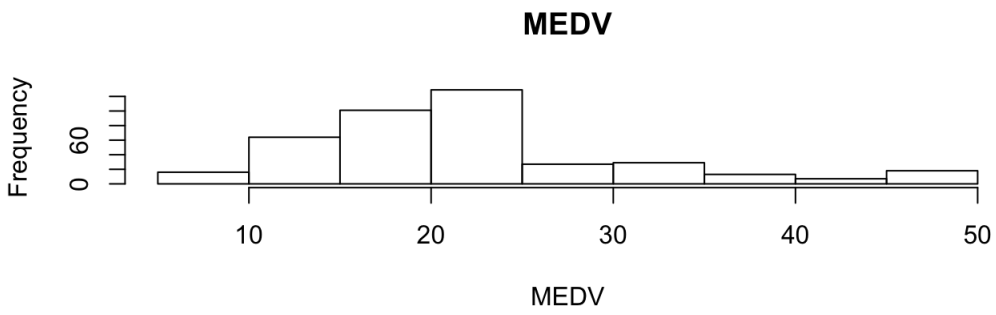
Exercise 1 : Boston Housing Dataset

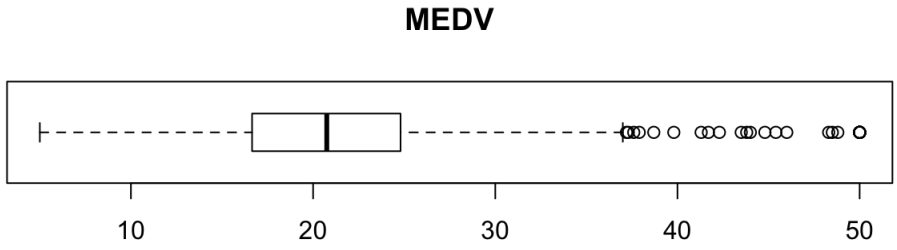
Load and explore the dataset

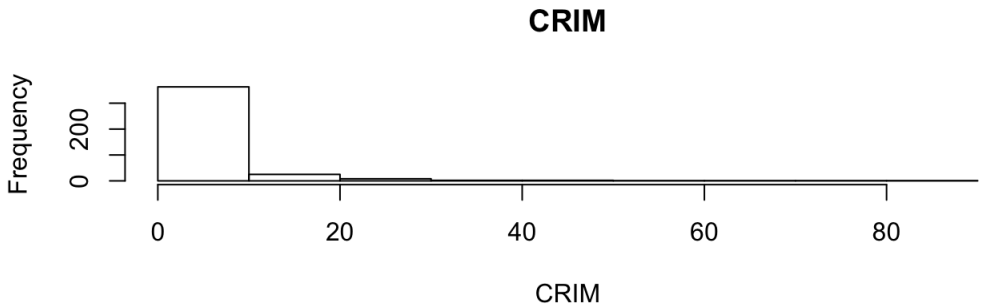
```
rm(list=ls())
boston<-dataset_boston_housing()
train_x<-boston$train$x
train_y<-boston$train$y
test_x<-boston$test$x
test_y<-boston$test$y

sum(is.na(c(train_x, train_y, test_x, test_y)))
```

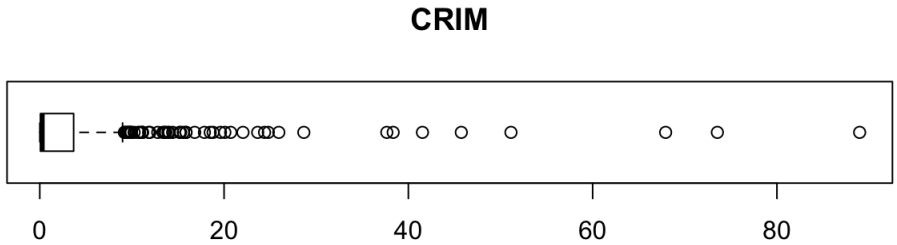
```
## [1] 0
```

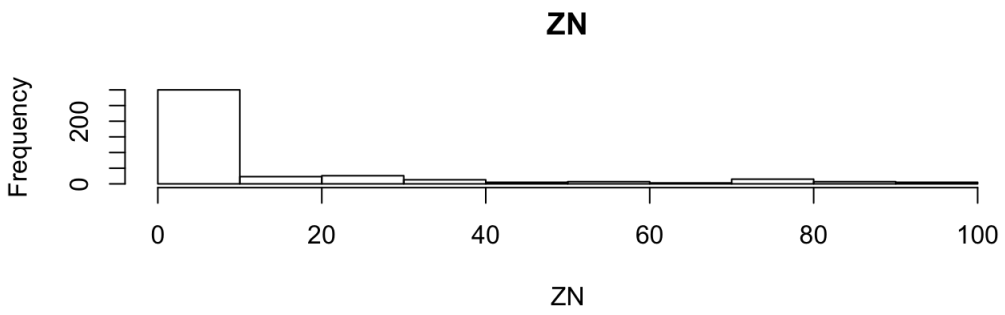
```
colnames(train_x)<-c("CRIM",
                     "ZN",
                     "INDUS",
                     "CHAS",
                     "NOX",
                     "RM",
                     "AGE",
                     "DIS",
                     "RAD",
                     "TAX",
                     "PTRATIO",
                     "B",
                     "LSTAT")

colnames(test_x)<-c("CRIM",
                    "ZN",
                    "INDUS",
                    "CHAS",
                    "NOX",
                    "RM",
                    "AGE",
                    "DIS",
                    "RAD",
                    "TAX",
                    "PTRATIO",
                    "B",
                    "LSTAT")


## Creating dataframes for aim of predict later
train_y_df<-data.frame(matrix(unlist(train_y), nrow=length(train_y), byrow=T))
train_y_df <- `colnames<-`(train_y_df, "MEDV")

train_x_df<-as.data.frame(unlist(train_x))

test_y_df<-data.frame(matrix(unlist(test_y), nrow=length(test_y), byrow=T))
test_y_df <- `colnames<-`(test_y_df, "MEDV")

test_x_df<-as.data.frame(unlist(test_x))

training_df<-data.frame(cbind(train_y_df,train_x_df))
test_df<-data.frame(cbind(test_y_df,test_x_df))

par(mfrow=c(2,1))
for (i in 1:14) {
  hist(training_df[,i],main=names(training_df)[i],xlab=names(training_df)[i])
  boxplot(training_df[,i],main=names(training_df)[i], horizontal = T)

}
```

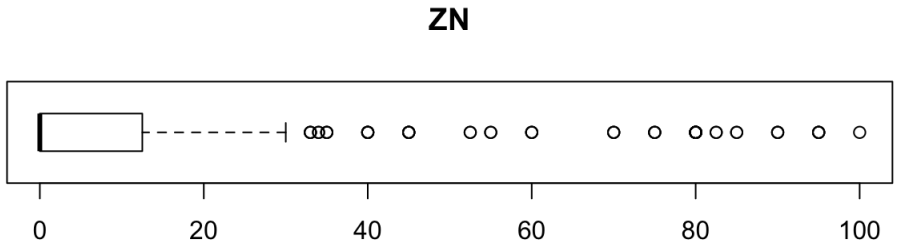

MEDV
Frequency
0
60
10
20
30
40
50
MEDV

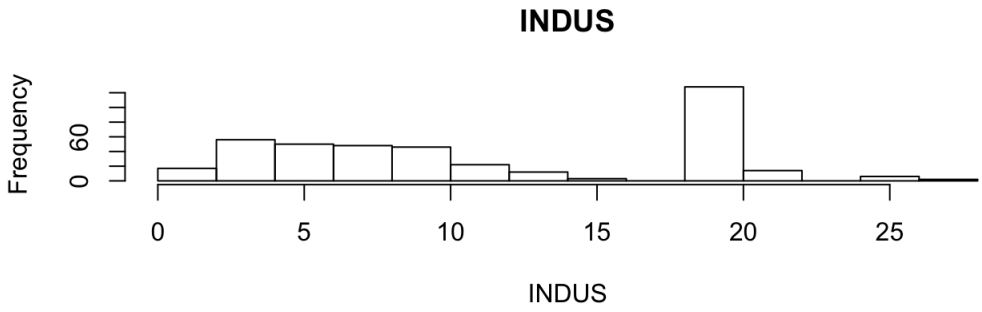

MEDV
10
20
30
40
50

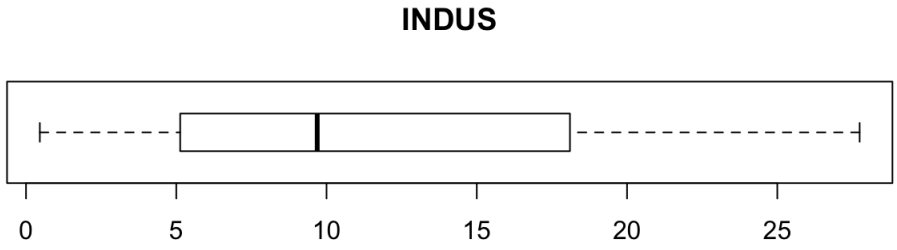

CRIM
Frequency
0
200
0
20
40
60
80
CRIM

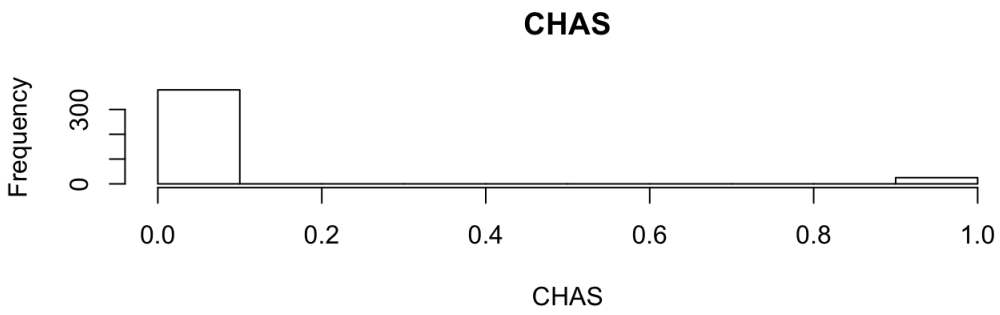

CRIM
0
20
40
60
80

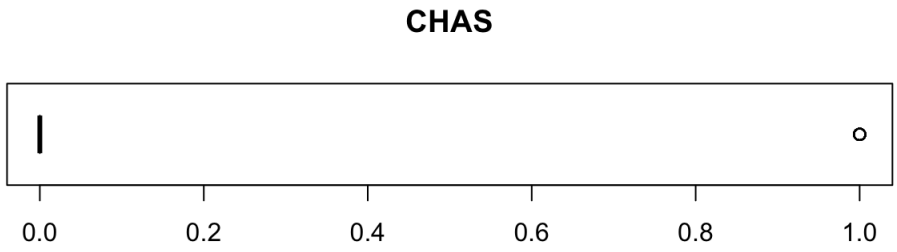

ZN
Frequency
0
200
0
20
40
60
80
100
ZN

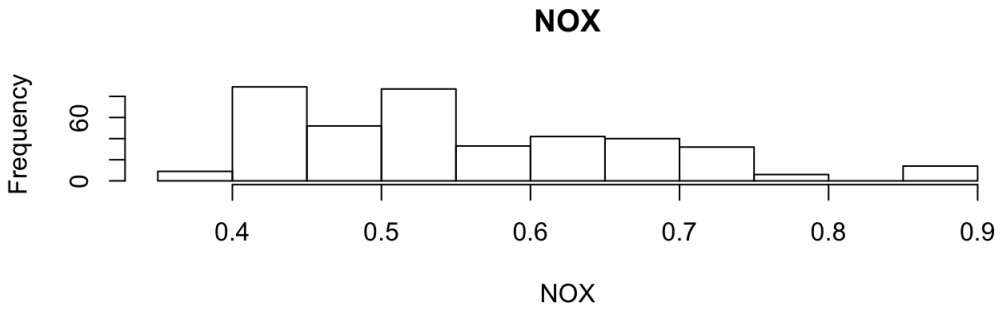

ZN
0
20
40
60
80
100

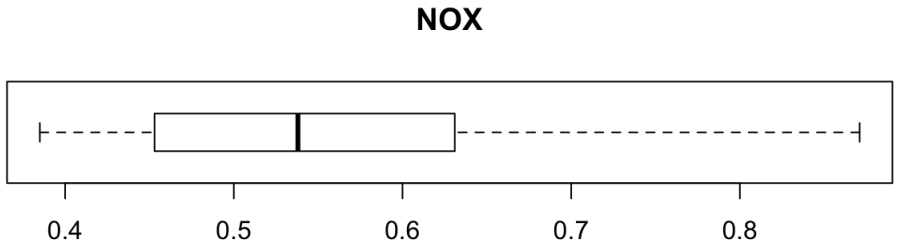

INDUS
Frequency
0
60
0
5
10
15
20
25
INDUS

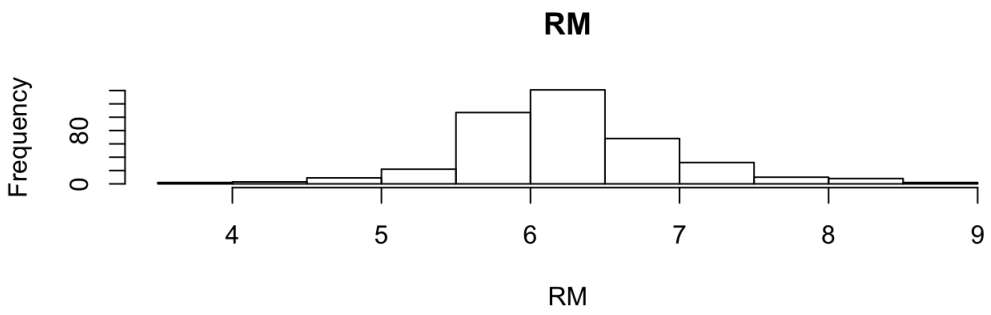

INDUS
0
5
10
15
20
25

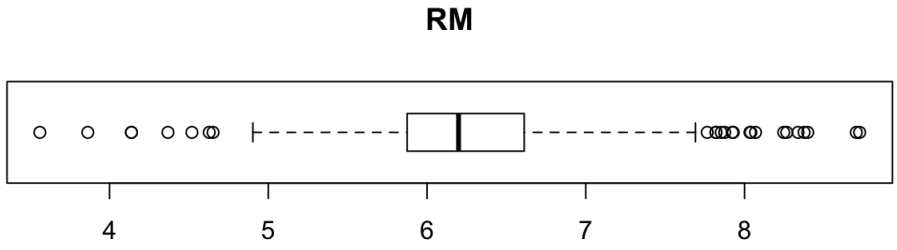

CHAS
Frequency
0
300
0.0
0.2
0.4
0.6
0.8
1.0
CHAS

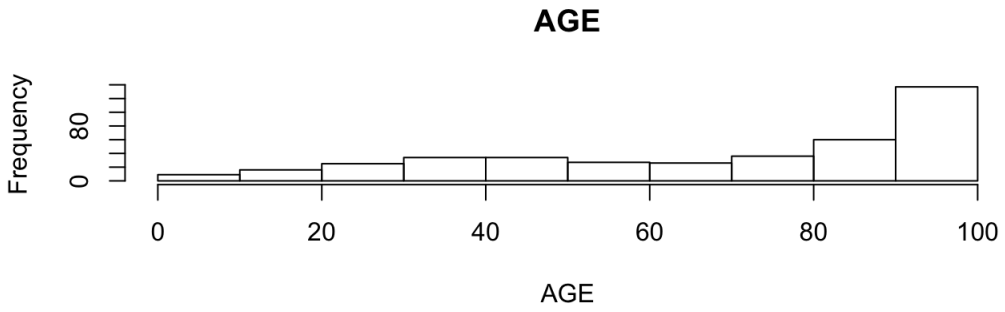

CHAS
0.0
0.2
0.4
0.6
0.8
1.0

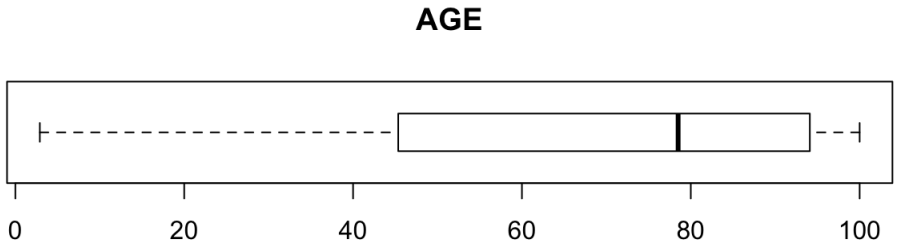

NOX
Frequency
0
60
0.4
0.5
0.6
0.7
0.8
0.9
NOX

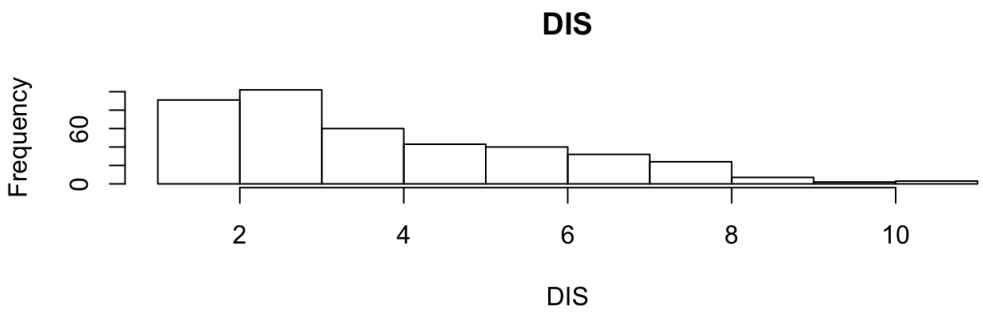

NOX
0.4
0.5
0.6
0.7
0.8

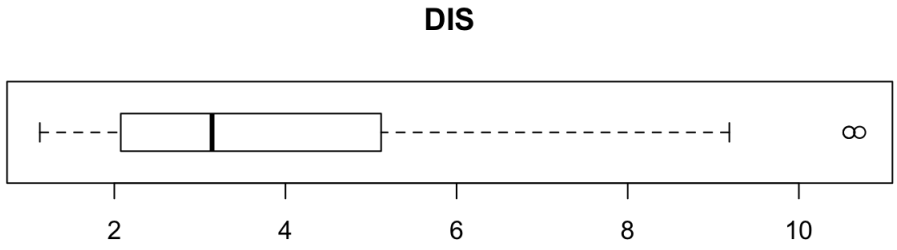

RM
Frequency
0
80
4
5
6
7
8
9
RM

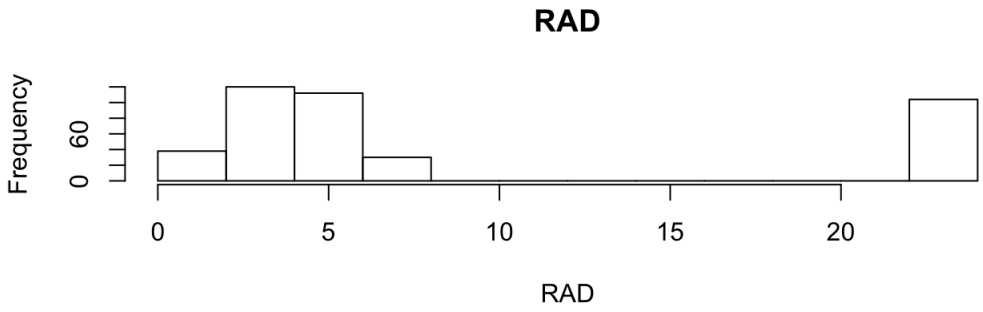

RM
4
5
6
7
8

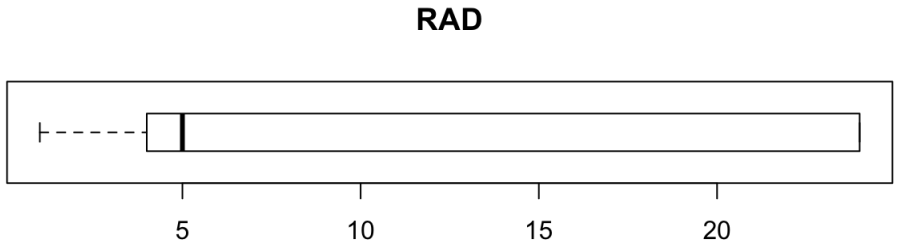

AGE
Frequency
0
80
0
20
40
60
80
100
AGE

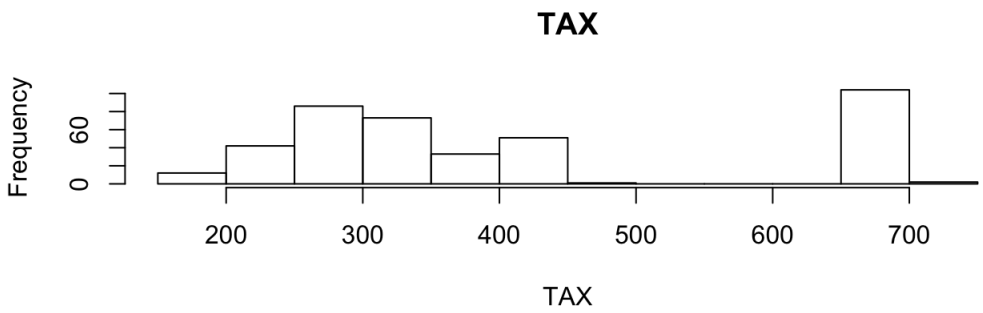

AGE
0
20
40
60
80
100

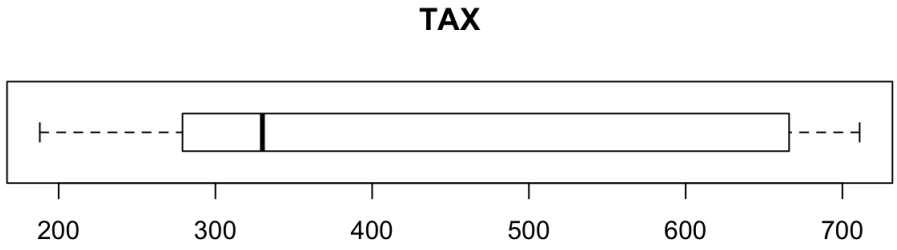

DIS
Frequency
0
60
2
4
6
8
10
DIS

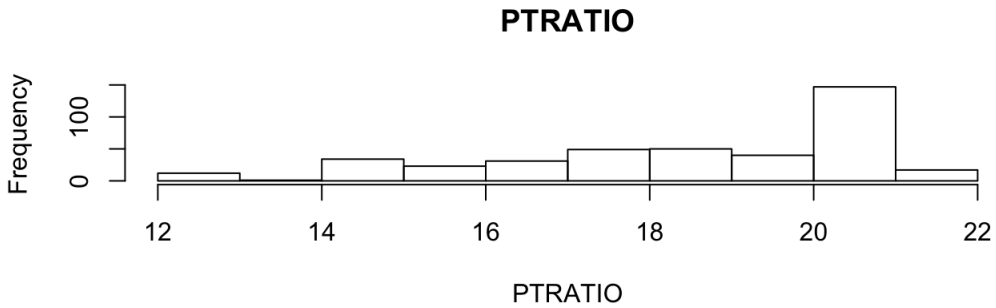

DIS
2
4
6
8
10

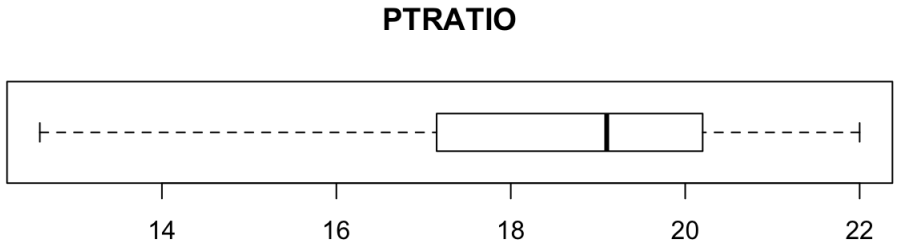

RAD
Frequency
0
60
0
5
10
15
20
RAD

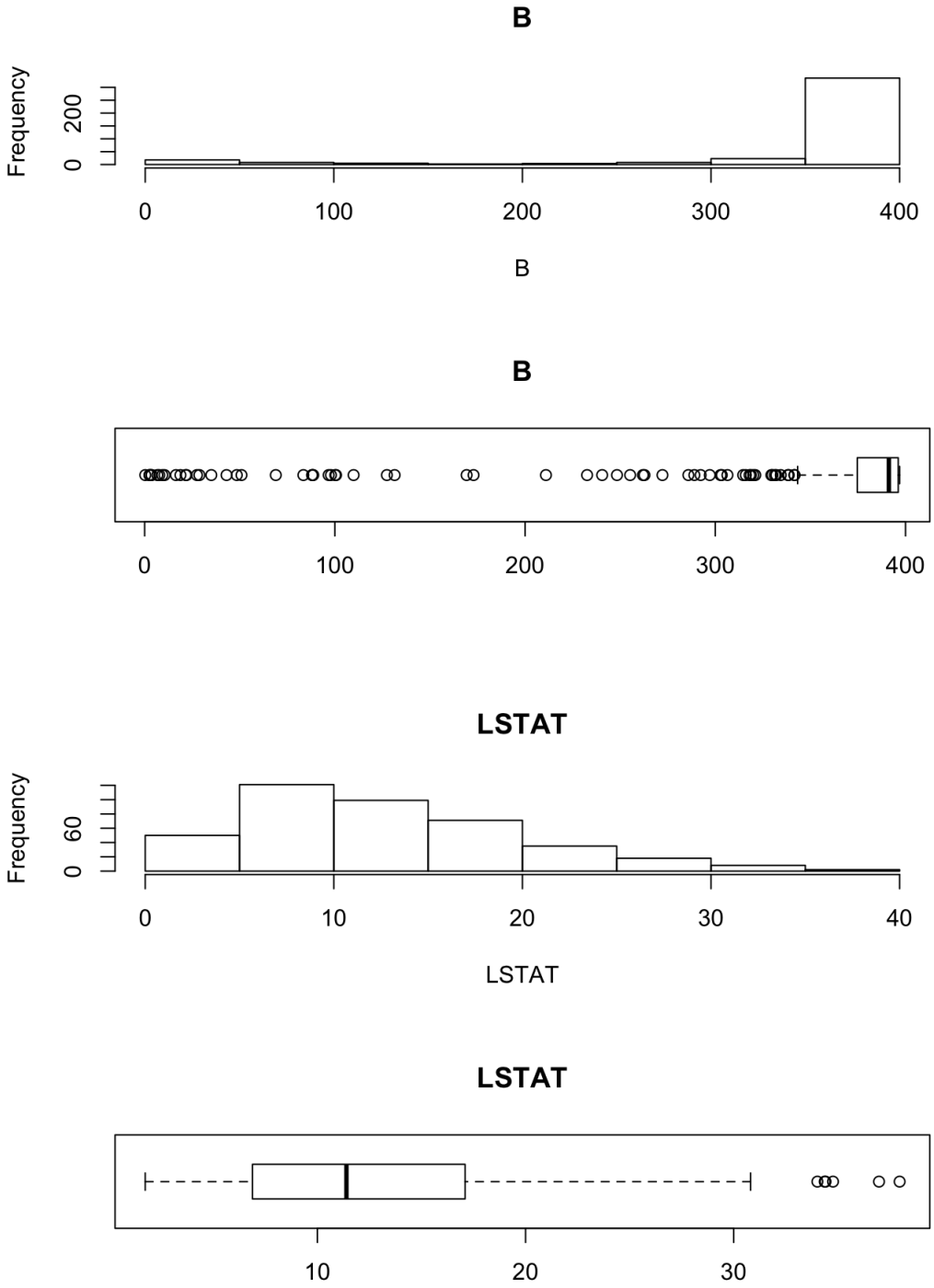

RAD
5
10
15
20


TAX
Frequency
0
60
200
300
400
500
600
700
TAX


TAX
200
300
400
500
600
700

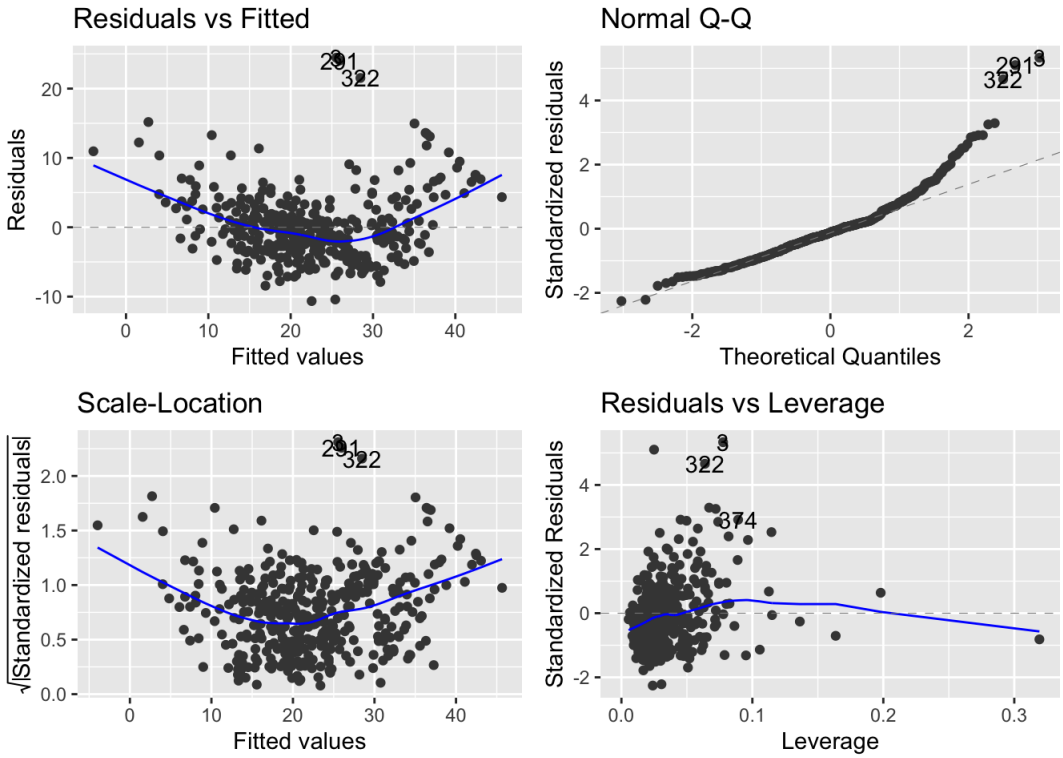

PTRATIO
Frequency
0
100
12
14
16
18
20
22
PTRATIO


PTRATIO
14
16
18
20
22



1) Fit a multivariate linear regression and show the diagnostic plots.

```
## Model fit
mod_lm<-lm(MEDV ~., data=training_df)

par(mfrow=c(2,2))
autoplot(mod_lm)
```



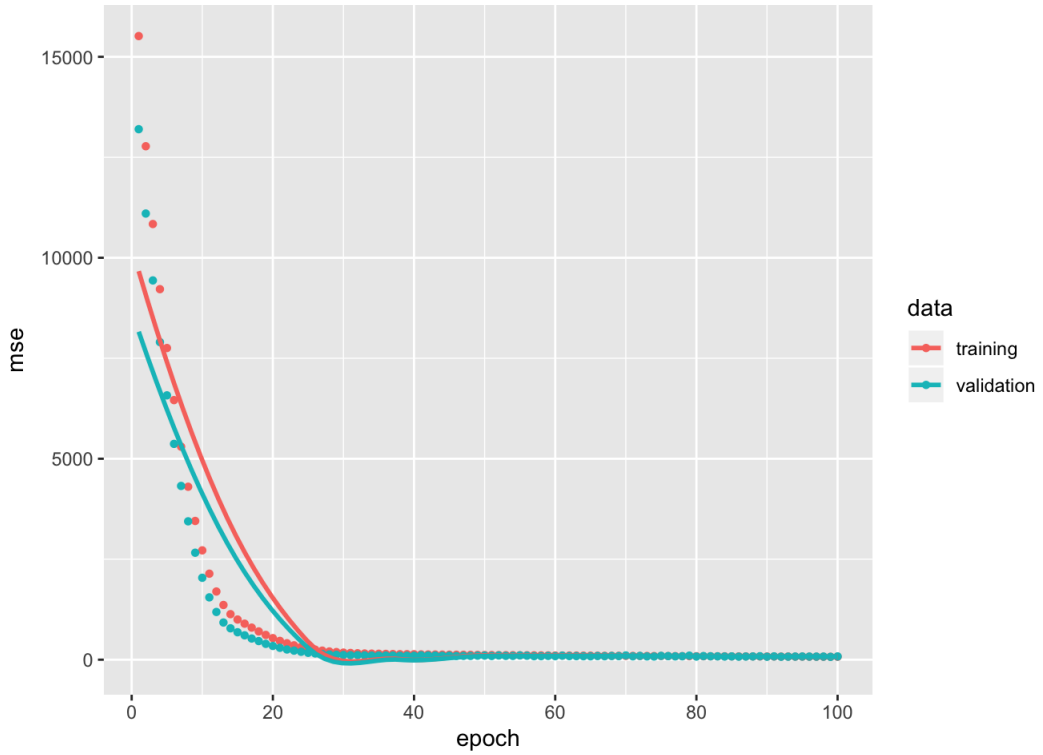
None of the OLS assumptions is met.

2) Replicate this model using a neural network mindset by using Keras.

```
## Create
mod_nn0<-keras_model_sequential()

mod_nn0 %>%
  layer_dense(units = 1, input_shape=c(13))

## Compile
mod_nn0 %>%
  compile(loss="mean_squared_error", optimizer = optimizer_rmsprop(), metric="mse")

summary(mod_nn0)
```

```
## Model: "sequential"
## ____________________________________________________________________________
## Layer (type)                     Output Shape                  Param #
## ============================================================================
## dense (Dense)                    (None, 1)                     14
## ============================================================================
## Total params: 14
## Trainable params: 14
## Non-trainable params: 0
## ____________________________________________________________________________
```

```
## Fit
set.seed(1)
fitted_nn0<- mod_nn0 %>% fit((train_x),
                             (train_y),
                             batch_size = 32,
                             epochs = 100,
                             validation_split = 0.25,
                             verbose = 2)

plot(fitted_nn0, metrics = "mse")
```



3) Retrieve the associated weights, and compute the performance on both sets.

```
weights0<- mod_nn0 %>% get_weights()
weights_mat0<-as.matrix(rbind(weights0[[2]][1],matrix(weights0[[1]][1:13,1],ncol=1)), byRow=T)
weights_mat0
```

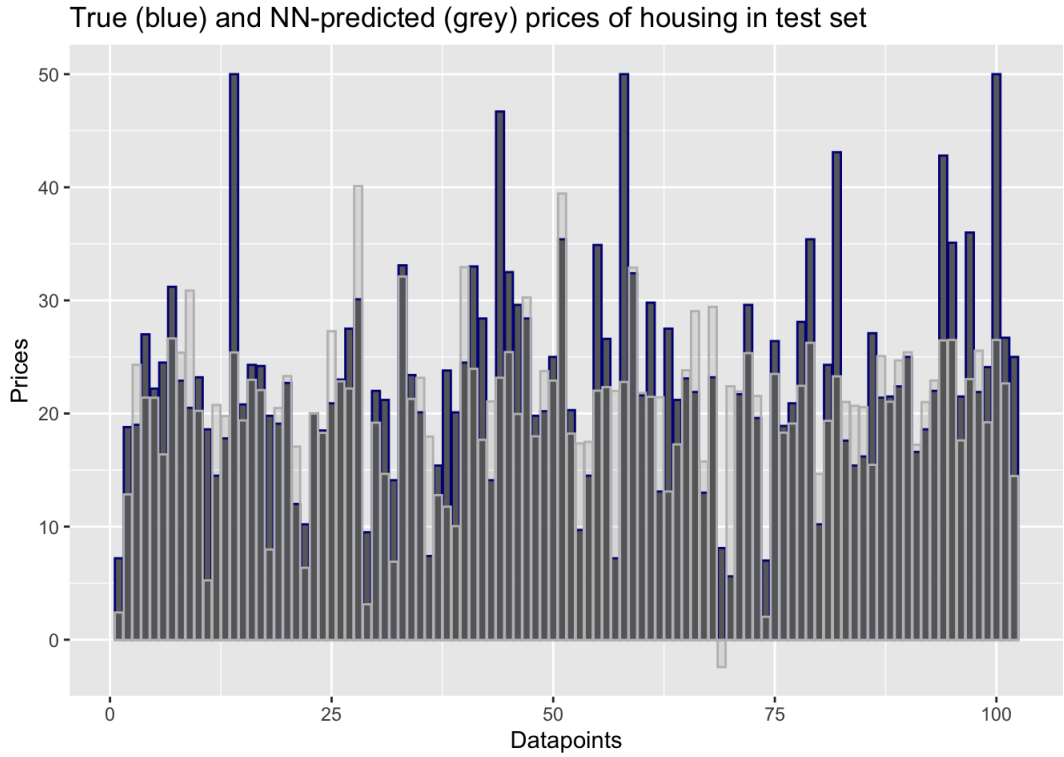
```
##              [,1]
##  [1,]  0.33092600
##  [2,]  0.38431489
##  [3,]  0.19440871
##  [4,]  0.54240590
##  [5,]  0.67306298
##  [6,] -0.16532634
##  [7,]  0.39121684
##  [8,]  0.02191874
##  [9,] -0.25725791
## [10,]  0.63004839
## [11,] -0.05196549
## [12,]  0.77636868
## [13,]  0.04374481
## [14,] -0.55781555
```

```
## Bias weight (b0), and usual neurons weights (bp)

## Performances
performance0_train <- mod_nn0 %>% evaluate(train_x, train_y)
performance0_test <- mod_nn0 %>% evaluate(test_x, test_y)

rbind(performance0_train, performance0_test)
```

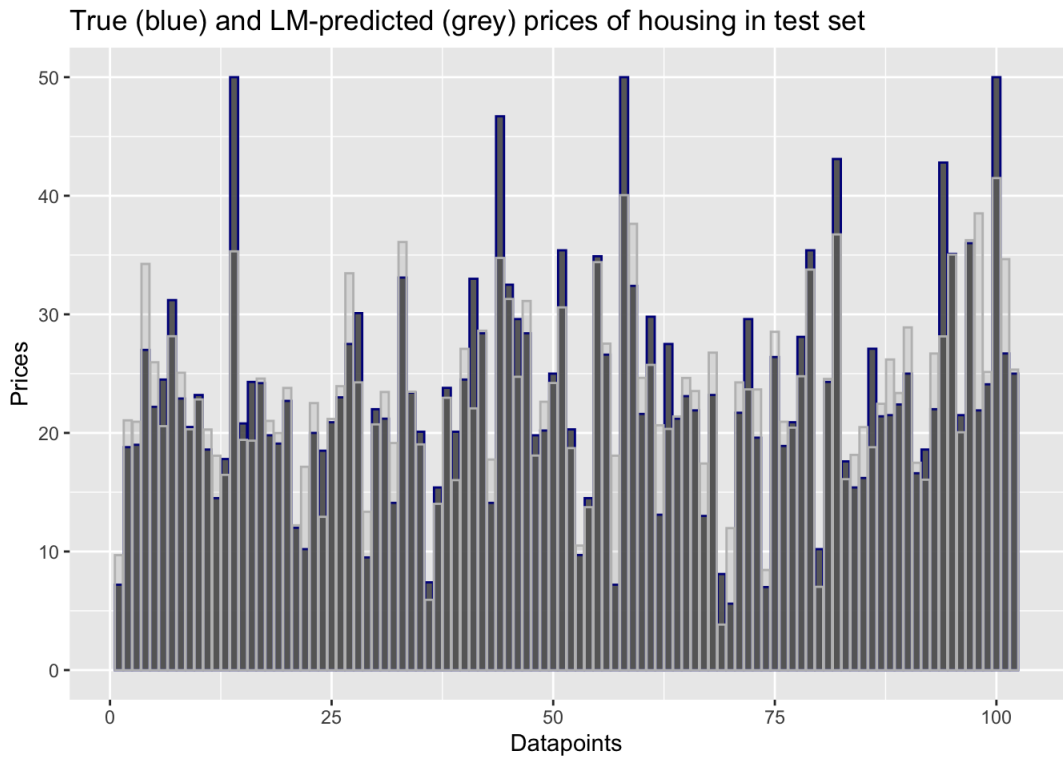
```
##                    loss     mse
## performance0_train 72.97604 72.97604
## performance0_test  70.89266 70.89266
```

4) Compute y_hat using the features and the weights and compare the results with the prediction of the linear regression.

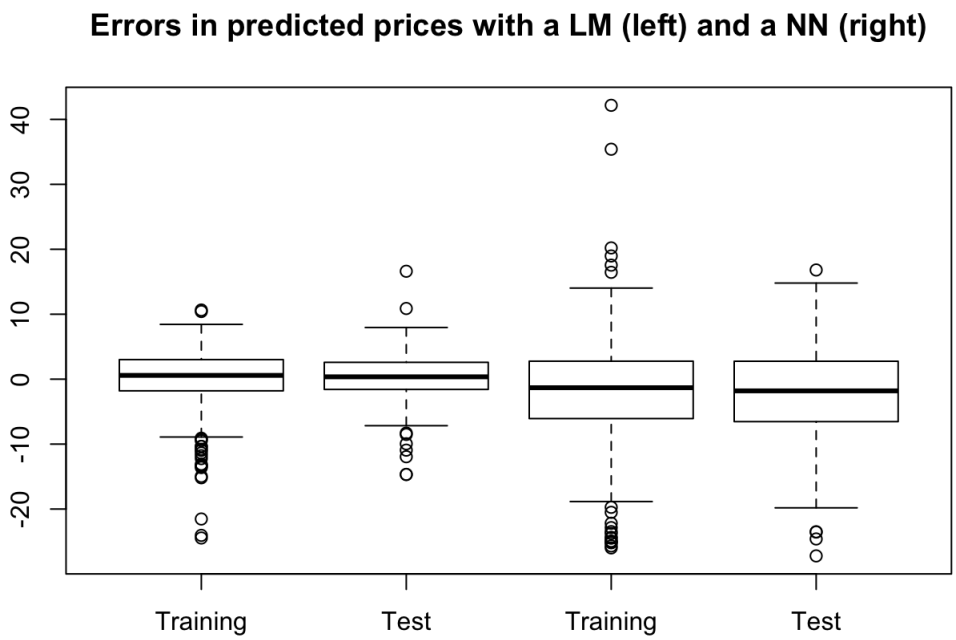
```
## Training
train0_mat_1<-matrix(rep(1,each=404), ncol=1, byrow=TRUE)
predict0_train_mat<- cbind(train0_mat_1,train_x) %*% weights_mat0
rmse_nn1_train <- sqrt(mean((predict0_train_mat[,1]-train_y)^2))

## Validation
test0_mat_1<-matrix(rep(1,each=102), ncol=1, byrow=TRUE)
predict0_test_mat<-cbind(test0_mat_1, test_x) %*% weights_mat0
rmse_nn1_test <- sqrt(mean((predict0_test_mat[,1]-test_y)^2))

## Plot of NN fitted vs true in the test
ggplot() + geom_col(aes(seq(1,102),test_y), color = "darkblue", alpha=0.9) +
  geom_col(aes(seq(1,102),predict0_test_mat[,1]), color = "grey", alpha = 0.1) +
  labs(title="True (blue) and NN-predicted (grey) prices of housing in test set", x="Datapoints", y="Prices"
)
```




```
## Plot of LM fitted vs true in the test
ggplot() + geom_col(aes(seq(1,102),test_y), color = "darkblue", alpha=0.9) +
  geom_col(aes(seq(1,102),predict(mod_lm, data.frame(test_x))), color = "grey", alpha = 0.1) +
  labs(title="True (blue) and LM-predicted (grey) prices of housing in test set", x="Datapoints", y="Prices"
)
```




```
## Comparison of results with linear regression

## Training
rmse_lm_train<-sqrt(mean((fitted(mod_lm)-train_y)^2))

## Validation
pred_lm<-predict.lm(mod_lm, test_x_df)
rmse_lm_test<-sqrt(mean((pred_lm-test_y)^2))

par(mfrow=c(1,1))
boxplot((fitted(mod_lm)-train_y),
        (pred_lm-test_y),
        (predict0_train_mat[,1]-train_y),
        (predict0_test_mat[,1]-test_y),
        names = c("Training", "Test", "Training", "Test"),
        main = "Errors in predicted prices with a LM (left) and a NN (right)")
```



```
rbind(rmse_nn1_train, rmse_nn1_test, rmse_lm_train, rmse_lm_test)
```

```
##                    [,1]
## rmse_nn1_train 8.542601
## rmse_nn1_test  8.419778
## rmse_lm_train  4.690928
## rmse_lm_test   4.816181
```

The linear regression performs a lot better than the neural network with unscaled data.

5) Try normalizing the features before fitting the NN, and add two relu activated layers of 32 and 16 neurons. Show the number of parameters.

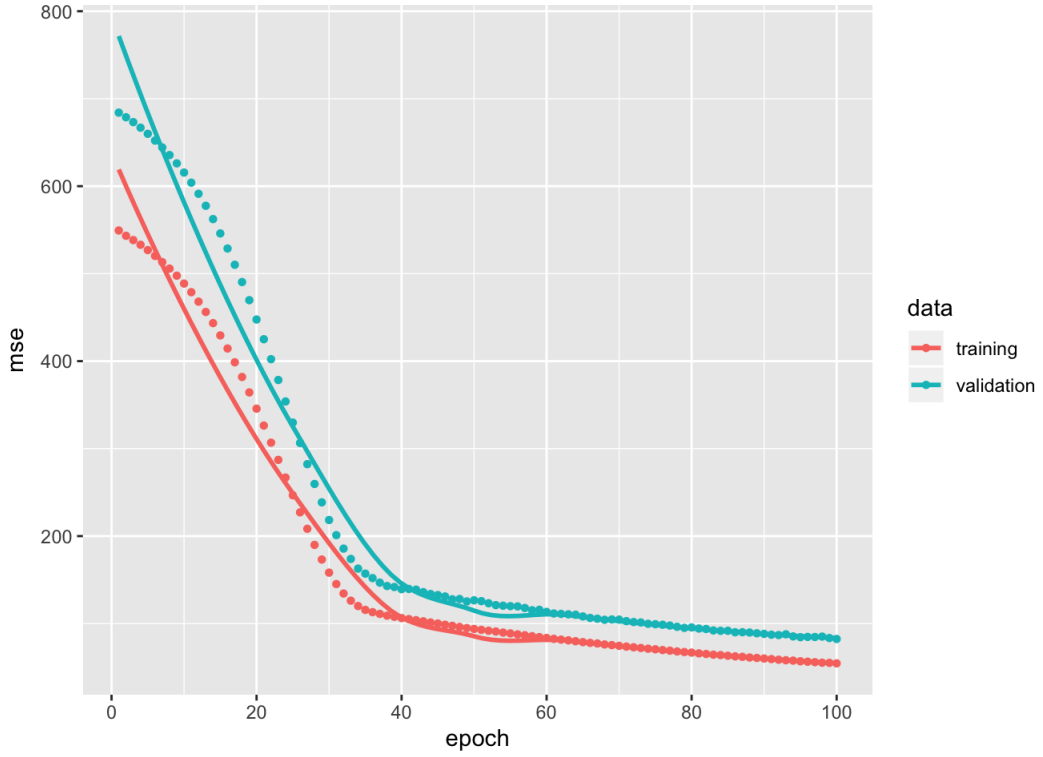
```
mod_nn1<-keras_model_sequential()

mod_nn1 %>%
  layer_dense(units = 1, input_shape=c(13)) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dense(units = 16, activation = "relu") %>%
  layer_dense(units = 1)

mod_nn1 %>%
compile(loss="mean_squared_error", optimizer = optimizer_rmsprop(), metric="mse")

summary(mod_nn1)
```

```
## Model: "sequential_1"
## ____________________________________________________________________________
## Layer (type)                     Output Shape                  Param #
## ============================================================================
## dense_1 (Dense)                  (None, 1)                     14
## ____________________________________________________________________________
## dense_2 (Dense)                  (None, 32)                    64
## ____________________________________________________________________________
## dense_3 (Dense)                  (None, 16)                    528
## ____________________________________________________________________________
## dense_4 (Dense)                  (None, 1)                     17
## ============================================================================
## Total params: 623
## Trainable params: 623
## Non-trainable params: 0
## ____________________________________________________________________________
```

We have 623 parameters.

6) Train your new NN and compute an appropriate performance metric on both sets.

```
## Scale
set.seed(1)
index_norm<-preProcess(train_x, method="range")
train_x_norm<-predict(index_norm,train_x)
test_x_norm<-predict(index_norm, test_x)

## Matrix transformation for keras
train_x_norm_mat<-as.matrix(train_x_norm)
test_x_norm_mat<-as.matrix(test_x_norm)

## Fit the model
set.seed(1)
fitted_nn1<- mod_nn1 %>% fit(train_x_norm_mat,
                             train_y,
                             batch_size = 32,
                             epochs = 100,
                             validation_split = 0.25,
                             verbose = 0)

plot(fitted_nn1, metrics = "mse")
```




```
performance1_train <- mod_nn1 %>% evaluate(train_x_norm, train_y)
performance1_test <- mod_nn1 %>% evaluate(test_x_norm, test_y)

rmse_nn2_train<-sqrt(performance1_train$mse)
rmse_nn2_test<-sqrt(performance1_test$mse)

rbind(rmse_nn1_train,
      rmse_nn1_test,
      rmse_lm_train,
      rmse_lm_test,
      rmse_nn2_train,
      rmse_nn2_test)
```

```
##                    [,1]
## rmse_nn1_train 8.542601
## rmse_nn1_test  8.419778
## rmse_lm_train  4.690928
## rmse_lm_test   4.816181
## rmse_nn2_train 7.822645
## rmse_nn2_test  7.838887
```

The second NN performs better than the first one.

Exercise 2 : Breast Cancer Wisconsin Dataset

Load and explore the data

```
## Load the data
rm(list=ls())
breast <- read.csv("wdbc.data", header = F)
sum(is.na(breast))
```

```
## [1] 0
```

```r
breast<-breast[,c(2:12)]
colnames(breast)<-c("Diagnosis",
                    "Radius",
                    "Texture",
                    "Perimeter",
                    "Area",
                    "Smoothness",
                    "Compactness",
                    "Concavity",
                    "Concave_points",
                    "Symmetry",
                    "Fractal_dimension")

## Transform the categorical "Diagnosis"

breast$Diagnosis<-ifelse(breast$Diagnosis=="M", 1,0)

#-----------------------------------------------------

## Explore the data
head(breast)
```

```
##   Diagnosis Radius Texture Perimeter   Area Smoothness Compactness
## 1         1  17.99   10.38    122.80 1001.0    0.11840     0.27760
## 2         1  20.57   17.77    132.90 1326.0    0.08474     0.07864
## 3         1  19.69   21.25    130.00 1203.0    0.10960     0.15990
## 4         1  11.42   20.38     77.58  386.1    0.14250     0.28390
## 5         1  20.29   14.34    135.10 1297.0    0.10030     0.13280
## 6         1  12.45   15.70     82.57  477.1    0.12780     0.17000
##   Concavity Concave_points Symmetry Fractal_dimension
## 1    0.3001        0.14710   0.2419           0.07871
## 2    0.0869        0.07017   0.1812           0.05667
## 3    0.1974        0.12790   0.2069           0.05999
## 4    0.2414        0.10520   0.2597           0.09744
## 5    0.1980        0.10430   0.1809           0.05883
## 6    0.1578        0.08089   0.2087           0.07613
```

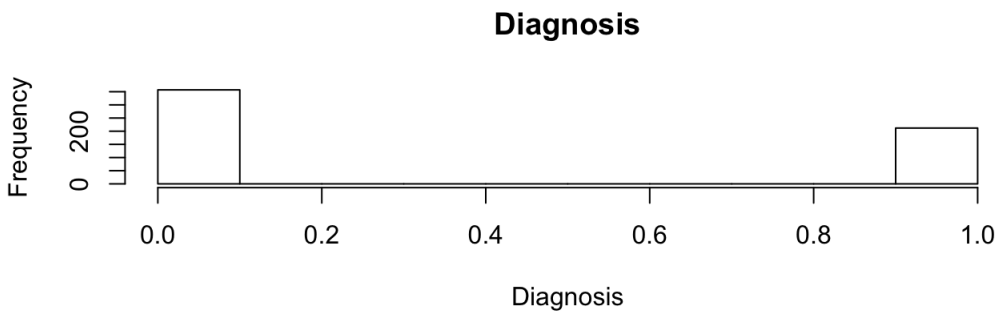
```r
str(breast)
```

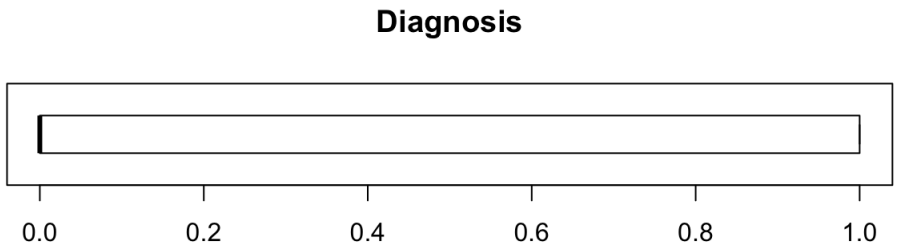
```
## 'data.frame':    569 obs. of  11 variables:
##  $ Diagnosis        : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Radius           : num  18 20.6 19.7 11.4 20.3 ...
##  $ Texture          : num  10.4 17.8 21.2 20.4 14.3 ...
##  $ Perimeter        : num  122.8 132.9 130 77.6 135.1 ...
##  $ Area             : num  1001 1326 1203 386 1297 ...
##  $ Smoothness       : num  0.1184 0.0847 0.1096 0.1425 0.1003 ...
##  $ Compactness      : num  0.2776 0.0786 0.1599 0.2839 0.1328 ...
##  $ Concavity        : num  0.3001 0.0869 0.1974 0.2414 0.198 ...
##  $ Concave_points   : num  0.1471 0.0702 0.1279 0.1052 0.1043 ...
##  $ Symmetry         : num  0.242 0.181 0.207 0.26 0.181 ...
##  $ Fractal_dimension: num  0.0787 0.0567 0.06 0.0974 0.0588 ...
```

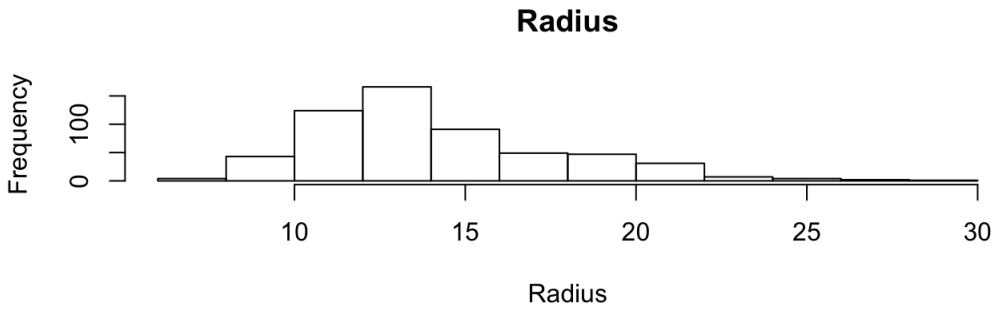
We only have numerical variables, with a binary outcome and no missing data.

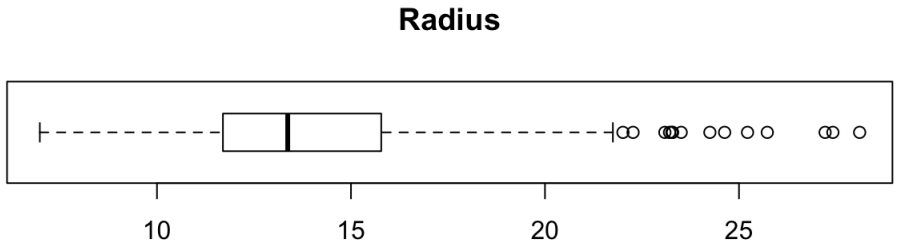
Let's look at the distribution of the datapoints.

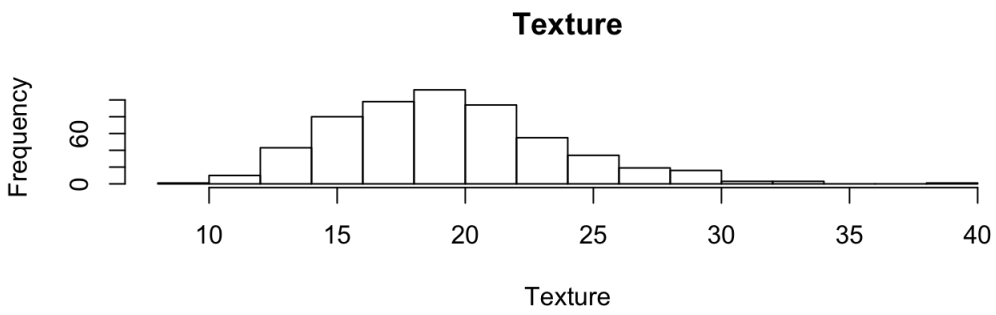
```r
par(mfrow=c(2,1))
for (i in 1:11) {
  hist(breast[,i],main=names(breast)[i],xlab=names(breast)[i])
  boxplot(breast[,i],main=names(breast)[i], horizontal = T)

}
```

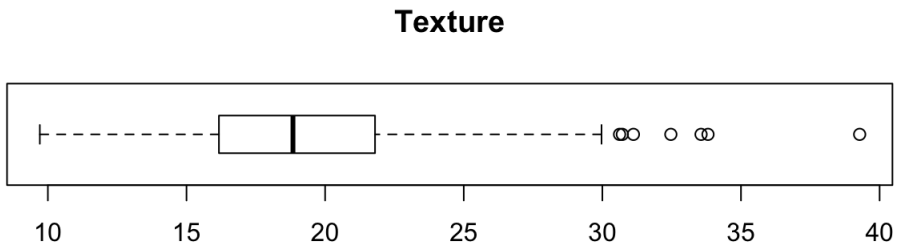

Diagnosis
Frequency
0
200
0.0
0.2
0.4
0.6
0.8
1.0
Diagnosis

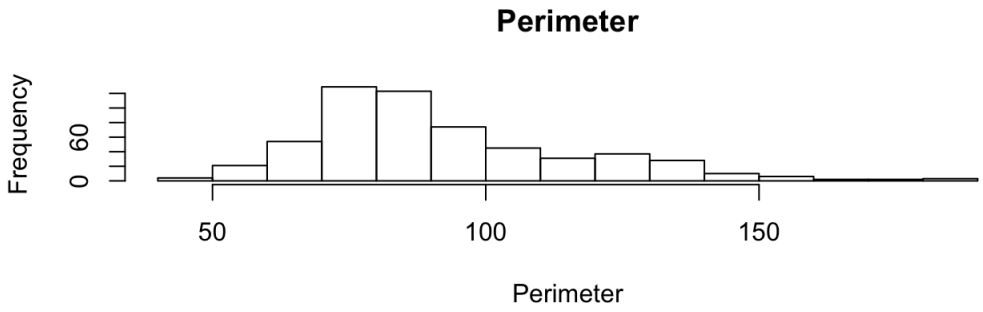

Diagnosis
0.0
0.2
0.4
0.6
0.8
1.0

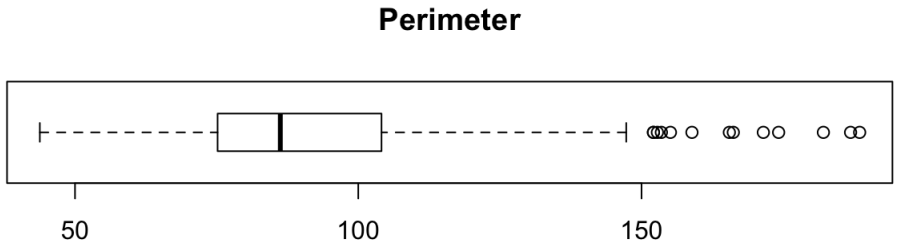

Radius
Frequency
0
100
10
15
20
25
30
Radius

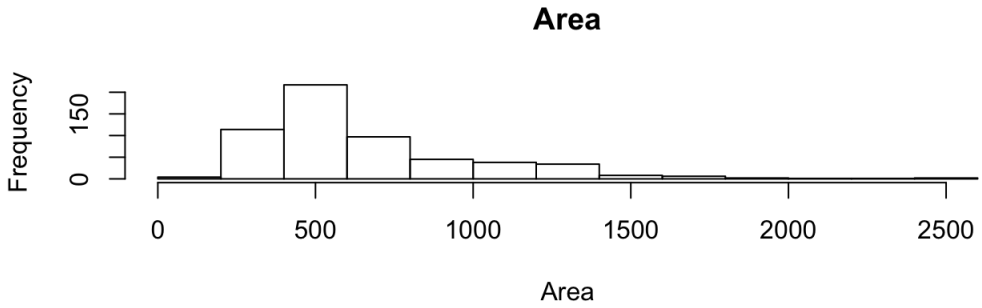

Radius
10
15
20
25

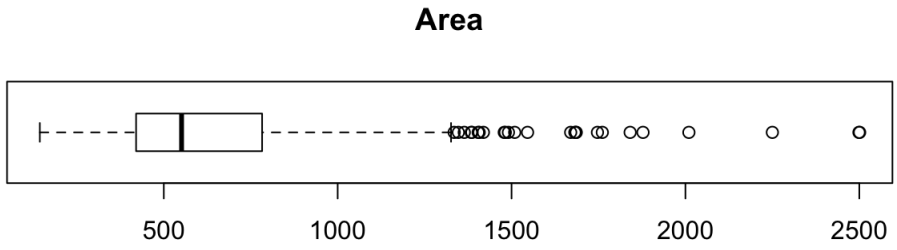

Texture
Frequency
0
60
10
15
20
25
30
35
40
Texture


Texture
10
15
20
25
30
35
40


Perimeter
Frequency
0
60
50
100
150
Perimeter

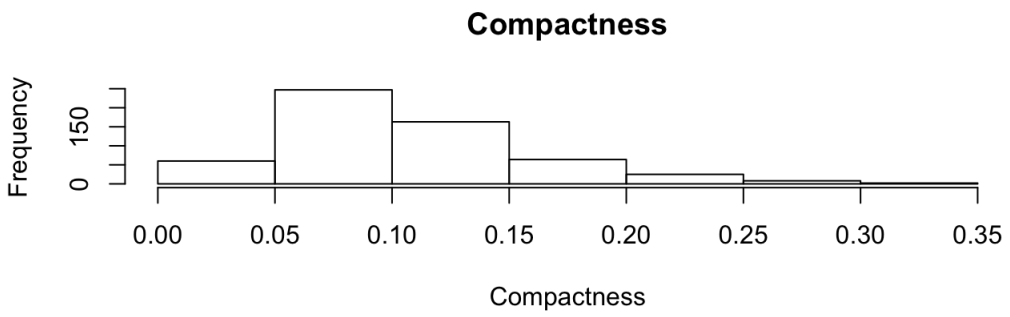

Perimeter
50
100
150

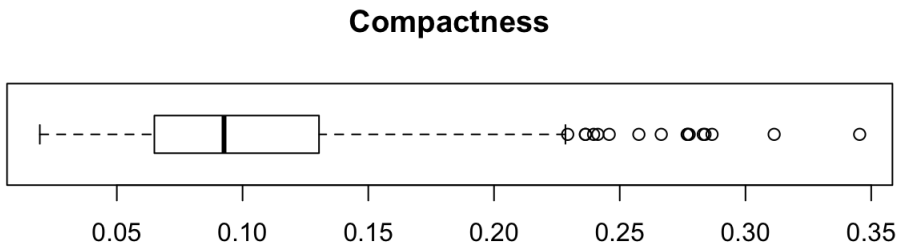

Area
Frequency
0
150
0
500
1000
1500
2000
2500
Area

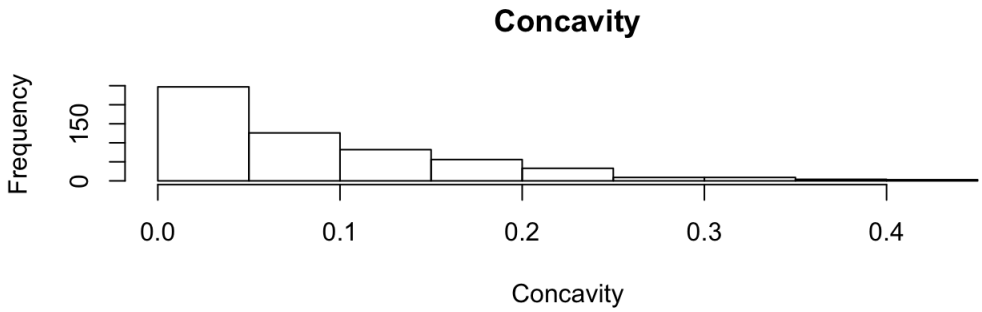

Area
500
1000
1500
2000
2500

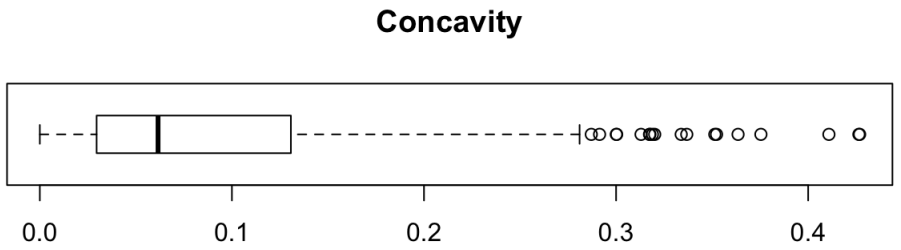
Smoothness
Frequency
0
100
0.06
0.08
0.10
0.12
0.14
0.16
Smoothness

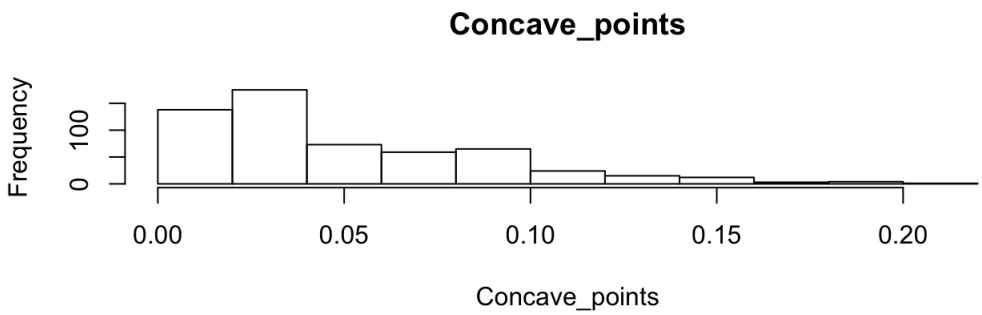
Smoothness
0.06
0.08
0.10
0.12
0.14
0.16

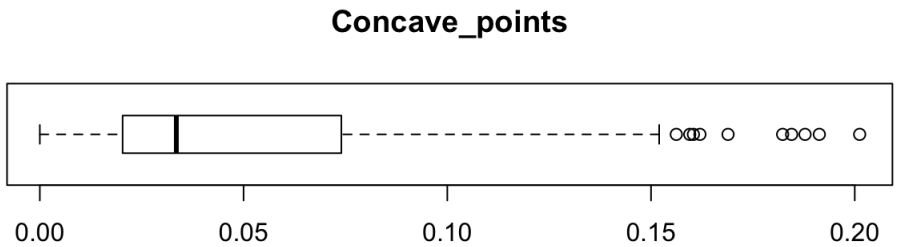

Compactness
Frequency
0
150
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Compactness

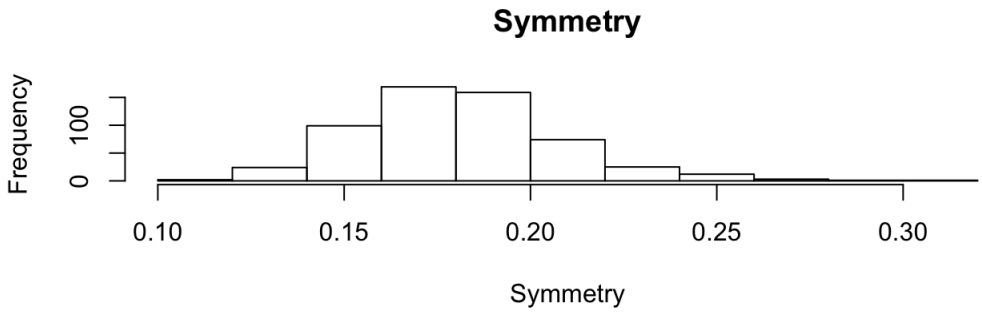

Compactness
0.05
0.10
0.15
0.20
0.25
0.30
0.35

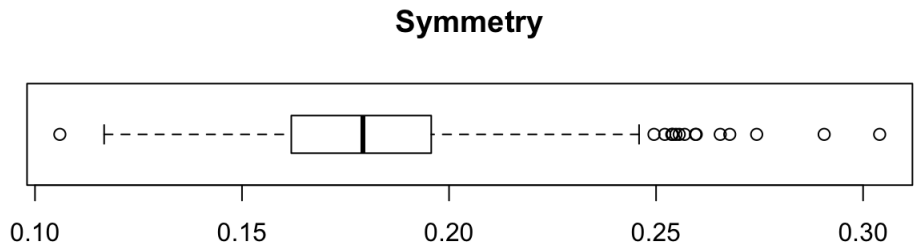

Concavity
Frequency
0
150
0.0
0.1
0.2
0.3
0.4
Concavity

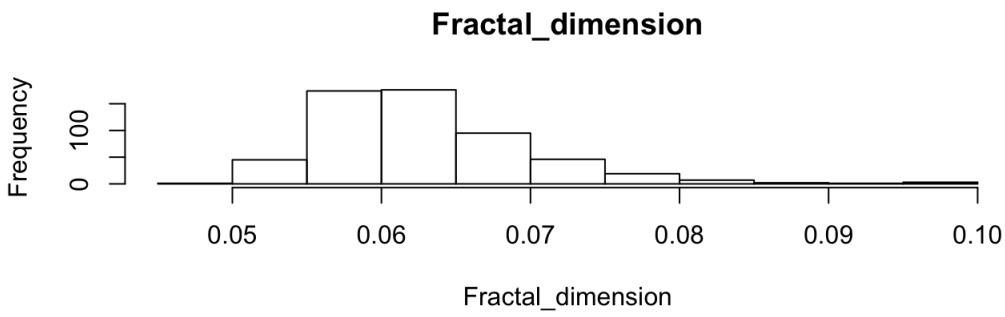

Concavity
0.0
0.1
0.2
0.3
0.4

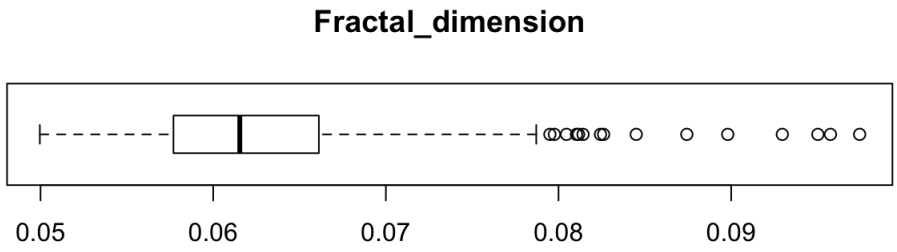

Concave_points
Frequency
0
100
0.00
0.05
0.10
0.15
0.20
Concave_points


Concave_points
0.00
0.05
0.10
0.15
0.20


Symmetry
Frequency
0
100
0.10
0.15
0.20
0.25
0.30
Symmetry


Symmetry
0.10
0.15
0.20
0.25
0.30






```
table(breast$Diagnosis)
```

```
##
##   0   1
## 357 212
```

First of all, the binary outcome variable isn't unbalanced.

Then, we see that the units of measure are not the same for each variable, thus we should partition the data and normalize the features before working with them.

1) Normalize the features

```
## 1) Partition 80%/20%
set.seed(1)
index<-createDataPartition(breast$Diagnosis,
                           0.8,
                           list=FALSE,
                           times=1)
training<-breast[index,]
validation<-breast[-index,]

## 2) Normalize
set.seed(1)
index_norm<-preProcess(training[,-1], method="range")
train_norm<-predict(index_norm,training)
val_norm<-predict(index_norm, validation)
```

2) Run a logistic regression

```
breast$Diagnosis<-as.factor(breast$Diagnosis)
mod_logit<-glm(Diagnosis ~ ., data = train_norm, family = binomial(link="logit"))
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(mod_logit)
```

```
##
## Call:
## glm(formula = Diagnosis ~ ., family = binomial(link = "logit"),
##     data = train_norm)
##
## Deviance Residuals:
##      Min        1Q    Median        3Q       Max
## -1.87745  -0.16121  -0.03995   0.00355   3.00518
##
## Coefficients:
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -11.7560     3.7065  -3.172  0.00152 **
## Radius            -106.4919    86.6211  -1.229  0.21892
## Texture              8.9773     1.7204   5.218 1.81e-07 ***
## Perimeter           51.0186    82.1320   0.621  0.53448
## Area               101.2149    43.0654   2.350  0.01876 *
## Smoothness           8.4618     4.0108   2.110  0.03488 *
## Compactness         -0.5482     7.0991  -0.077  0.93845
## Concavity            1.7896     3.7850   0.473  0.63634
## Concave_points      11.4419     6.3946   1.789  0.07357 .
## Symmetry             4.2496     2.2143   1.919  0.05496 .
## Fractal_dimension   -3.9136     4.4790  -0.874  0.38225
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 601.26  on 455  degrees of freedom
## Residual deviance: 116.71  on 445  degrees of freedom
## AIC: 138.71
##
## Number of Fisher Scoring iterations: 9
```

3) Predict the test set

```
pred_logit<-predict(mod_logit,
                    val_norm[,-1],
                    type="response")

pred_logit_class<-ifelse(pred_logit<0.5,0,1)

true_y<-head(val_norm$Diagnosis,20)
pred_glm_y<-head(pred_logit_class,20)
cbind(true_y, pred_glm_y)
```

```
##    true_y pred_glm_y
## 9       1          1
## 11      1          0
## 13      1          1
## 52      0          0
## 54      1          1
## 55      1          0
## 57      1          1
## 63      1          1
## 67      0          0
## 68      0          0
## 70      0          0
## 76      1          1
## 80      0          0
## 82      0          0
## 85      0          0
## 87      1          1
## 88      1          1
## 94      0          0
## 95      1          1
## 96      1          1
```

If we only look at the 20 first predictions, there are already two misclassified points.

4) Provide a confusion matrix and select an adapted measure of performance

```
confusionMatrix(as.factor(pred_logit_class),
                as.factor(val_norm$Diagnosis),
                positive="1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 69  6
##          1  1 37
##
##                Accuracy : 0.9381
##                  95% CI : (0.8765, 0.9747)
##     No Information Rate : 0.6195
##     P-Value [Acc > NIR] : 4.484e-15
##
##                   Kappa : 0.8656
##
##  Mcnemar's Test P-Value : 0.1306
##
##             Sensitivity : 0.8605
##             Specificity : 0.9857
##          Pos Pred Value : 0.9737
##          Neg Pred Value : 0.9200
##              Prevalence : 0.3805
##          Detection Rate : 0.3274
##    Detection Prevalence : 0.3363
##       Balanced Accuracy : 0.9231
##
##        'Positive' Class : 1
##
```

```
conf_logit<-confusionMatrix(as.factor(pred_logit_class),
                as.factor(val_norm$Diagnosis),
                positive="1")
```

The ‘accuracy’ and ‘balanced accuracy’ are high enough. The class ‘0’ is however better classified than the ‘1’.

5) Replicate the regression using a neural network mindset

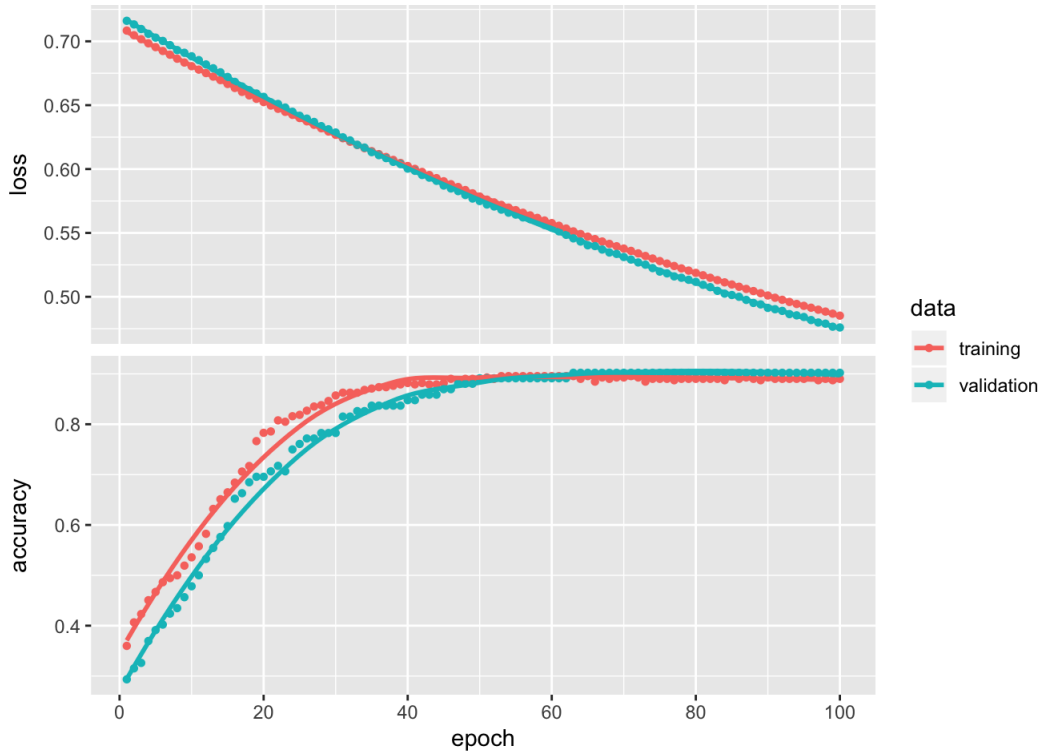
```
set.seed(1)
mod_nn<-keras_model_sequential()

set.seed(1)
mod_nn %>%
  layer_dense(units=1, input_shape=c(10), activation="sigmoid")

set.seed(1)
mod_nn %>%
  compile(loss="binary_crossentropy", optimizer = optimizer_rmsprop(), metric="accuracy")

summary(mod_nn)
```

```
## Model: "sequential_2"
## ____________________________________________________________________________
## Layer (type)                     Output Shape                  Param #
## ============================================================================
## dense_5 (Dense)                  (None, 1)                     11
## ============================================================================
## Total params: 11
## Trainable params: 11
## Non-trainable params: 0
## ____________________________________________________________________________
```

We have 11 parameters.

6) Fit the NN model, with at least 100 epochs

```
x_train_mat<-as.matrix((train_norm)[,2:11])
y_train_mat<-as.matrix((train_norm)[,1])
x_val_mat<-as.matrix((val_norm)[,2:11])
y_val_mat<-as.matrix((val_norm)[,1])

set.seed(1)
fitted_nn<- mod_nn %>% fit(x_train_mat,
                           y_train_mat,
                           batch_size = 32,
                           epochs = 100,
                           validation_split = 0.2,
                           verbose = 1)

plot(fitted_nn)
```




```
performance_train <- mod_nn %>% evaluate(x_train_mat, y_train_mat)

performance_test <- mod_nn %>% evaluate(x_val_mat, y_val_mat)

performance_train
```

```
## $loss
## [1] 0.4826419
##
## $accuracy
## [1] 0.8925439
```

```
performance_test
```

```
## $loss
## [1] 0.4873679
##
## $accuracy
## [1] 0.9292035
```

7) Predict y_hat using the weights and the feature values and compare these results with the ones we got from the NN model

```r
## Extract the weights : bias (b0) and weights (bp)
weights <- mod_nn %>% get_weights()
weights_mat<-as.matrix(rbind(weights[[2]][1],matrix(weights[[1]][1:10,1],ncol=1)), byRow=T)
weights_mat
```

```
##              [,1]
##  [1,] -0.88759261
##  [2,]  0.47194266
##  [3,] -0.10361855
##  [4,]  1.05037105
##  [5,]  0.78077465
##  [6,] -0.40871787
##  [7,]  0.26728153
##  [8,]  0.76569623
##  [9,]  1.24771404
## [10,] -0.55688113
## [11,] -0.07401528
```

```r
## Prediction of training set
train_mat_1<-matrix(rep(1,each=456), ncol=1, byrow=TRUE)

predict_train_mat<-cbind(train_mat_1,x_train_mat) %*% weights_mat

proba_train <- exp(predict_train_mat)/(1+exp(predict_train_mat))

predict_train_class<-ifelse(proba_train<0.5,0,1)


## Prediction of test set
val_mat_1<-matrix(rep(1,each=113), ncol=1, byrow=TRUE)

predict_val_mat<-cbind(val_mat_1, x_val_mat) %*% weights_mat

proba_test <- exp(predict_val_mat)/(1+exp(predict_val_mat))

predict_val_class<-ifelse(proba_test<0.5,0,1)

## Comparison: Show the 20 first model predictions and computed predictions
mod_nn_pred <- mod_nn %>% predict_classes(x_val_mat)
pred_mod_nn_20 <- head(mod_nn_pred, 20)
pred_val_class_20 <- head(predict_val_class, 20)
cbind(pred_mod_nn_20, pred_val_class_20)
```

```
##    [,1] [,2]
## 9     1    1
## 11    0    0
## 13    1    1
## 52    0    0
## 54    1    1
## 55    0    0
## 57    1    1
## 63    1    1
## 67    0    0
## 68    0    0
## 70    0    0
## 76    1    1
## 80    0    0
## 82    0    0
## 85    0    0
## 87    0    0
## 88    1    1
## 94    0    0
## 95    1    1
## 96    1    1
```

The predictions are similar.

Comparison with the true y, the glm predictions and the NN ones.

```
cbind(pred_glm_y, pred_val_class_20, true_y)
```

```
##    pred_glm_y   true_y
## 9           1 1      1
## 11          0 0      1
## 13          1 1      1
## 52          0 0      0
## 54          1 1      1
## 55          0 0      1
## 57          1 1      1
## 63          1 1      1
## 67          0 0      0
## 68          0 0      0
## 70          0 0      0
## 76          1 1      1
## 80          0 0      0
## 82          0 0      0
## 85          0 0      0
## 87          1 0      1
## 88          1 1      1
## 94          0 0      0
## 95          1 1      1
## 96          1 1      1
```

The NN misclassifies a ‘1’.

8) Provide a confusion matrix for the test set

```
confusionMatrix(as.factor(predict_val_class), as.factor(y_val_mat), positive="1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 68  6
##          1  2 37
##
##                Accuracy : 0.9292
##                  95% CI : (0.8653, 0.9689)
##     No Information Rate : 0.6195
##     P-Value [Acc > NIR] : 3.716e-14
##
##                   Kappa : 0.8471
##
##  Mcnemar's Test P-Value : 0.2888
##
##             Sensitivity : 0.8605
##             Specificity : 0.9714
##          Pos Pred Value : 0.9487
##          Neg Pred Value : 0.9189
##              Prevalence : 0.3805
##          Detection Rate : 0.3274
##    Detection Prevalence : 0.3451
##       Balanced Accuracy : 0.9159
##
##        'Positive' Class : 1
##
```

It’s confirmed by the drop in the ‘accuracy’ metric from GLM to NN.

Exercise 3 : MNIST Dataset

Load the data

```
rm(list=ls())
mnist<-dataset_mnist()

x_train<-mnist$train$x
y_train<-mnist$train$y
x_test<-mnist$test$x
y_test<-mnist$test$y
```

1) Observe the dimensions of the dataset and plot some observations from the train set.

```
dimensions<-data.frame(cbind(dim(x_train),
                             dim(y_train),
                             dim(x_test),
                             dim(y_test)))

colnames(dimensions)<-c("x_train", "y_train", "x_test","y_test")

dimensions
```

```
##   x_train y_train x_test y_test
## 1   60000   60000  10000  10000
## 2      28   60000     28  10000
## 3      28   60000     28  10000
```

```
## Plot some
par(mfrow = c(2,2))

n2 = mnist$train$x[200,,]
plot(as.raster(n2, max = 255))

n7 = mnist$train$x[092,,]
plot(as.raster(n7, max = 255))

n0 = mnist$train$x[002,,]
plot(as.raster(n0, max = 255))

n6 = mnist$train$x[450,,]
plot(as.raster(n6, max = 255))
```

2-3) Reshape and rescale both datasets for a DNN.

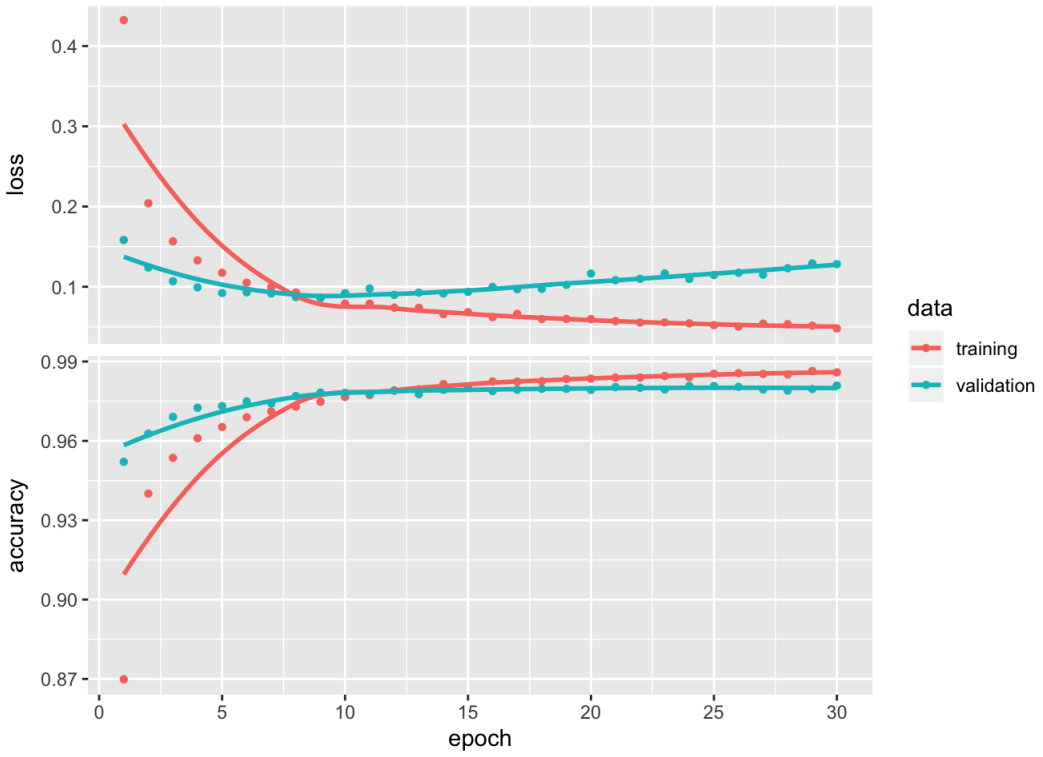
```
## Reshape
x_train <- array_reshape(x_train, c(nrow(x_train), 784))
x_test <- array_reshape(x_test, c(nrow(x_test), 784))

## Rescale : X values are ranging from 1 to 255
x_train <- x_train / 255
x_test <- x_test / 255

## Prepare the y's : one-hot encoding into binary class matrices
## They're ranging from 1 to 10
y_train <- to_categorical(y_train,10)
y_test <- to_categorical(y_test, 10)
```

4) Design a DNN.

```
mod_dnn<-keras_model_sequential()

mod_dnn %>%
  layer_dense(units = 256, activation = "relu", input_shape=c(784)) %>%
  layer_dropout(rate = 0.4) %>%
  layer_dense(units = 128, activation = "relu") %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 10, activation = "softmax")


mod_dnn %>% compile(
  loss = "categorical_crossentropy",
  optimizer = optimizer_rmsprop(),
  metrics = "accuracy"
)

summary(mod_dnn)
```

```
## Model: "sequential_3"
## ____________________________________________________________________________
## Layer (type)                     Output Shape                  Param #
## ============================================================================
## dense_6 (Dense)                  (None, 256)                   200960
## ____________________________________________________________________________
## dropout (Dropout)                (None, 256)                   0
## ____________________________________________________________________________
## dense_7 (Dense)                  (None, 128)                   32896
## ____________________________________________________________________________
## dropout_1 (Dropout)              (None, 128)                   0
## ____________________________________________________________________________
## dense_8 (Dense)                  (None, 10)                    1290
## ============================================================================
## Total params: 235,146
## Trainable params: 235,146
## Non-trainable params: 0
## ____________________________________________________________________________
```

```
set.seed(1)
fitted_dnn <- mod_dnn %>% fit(
  x_train,
  y_train,
  batch_size = 128,
  epochs = 30,
  validation_split = 0.2,
  verbose = 1
)

plot(fitted_dnn)
```



5) Provide a confusion matrix for both sets and compute the performance using an appropriate metric.

```
pred_train <- mod_dnn %>% predict_classes(x_train)
pred_test <- mod_dnn %>% predict_classes(x_test)

## Confusion matrix for training set
conf_train_dnn<-confusionMatrix(as.factor(pred_train), as.factor(mnist$train$y))
conf_train_dnn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1    2    3    4    5    6    7    8    9
##          0 5914    0    2    2    1    4    7    1    6    5
##          1    0 6718    3    0    8    1    0    5    3    3
##          2    2    4 5932   23    2    5    2   13   12    0
##          3    0    2    2 6067    0   12    0    1    8    6
##          4    1    3    3    1 5818    1    2    5    2   26
##          5    1    0    0   14    0 5374    6    1    9    5
##          6    0    3    3    0    2   12 5899    0    4    0
##          7    0    8    8    8    2    3    0 6237    2   20
##          8    3    4    5    7    0    4    2    0 5798    4
##          9    2    0    0    9    9    5    0    2    7 5880
##
## Overall Statistics
##
##                Accuracy : 0.994
##                  95% CI : (0.9933, 0.9946)
##     No Information Rate : 0.1124
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9933
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity           0.99848   0.9964  0.99564   0.9896  0.99589  0.99133
## Specificity           0.99948   0.9996  0.99883   0.9994  0.99919  0.99934
## Pos Pred Value        0.99529   0.9966  0.98949   0.9949  0.99249  0.99335
## Neg Pred Value        0.99983   0.9995  0.99952   0.9988  0.99956  0.99914
## Prevalence            0.09872   0.1124  0.09930   0.1022  0.09737  0.09035
## Detection Rate        0.09857   0.1120  0.09887   0.1011  0.09697  0.08957
## Detection Prevalence  0.09903   0.1124  0.09992   0.1016  0.09770  0.09017
## Balanced Accuracy     0.99898   0.9980  0.99724   0.9945  0.99754  0.99534
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity           0.99679   0.9955  0.99094  0.98840
## Specificity           0.99956   0.9991  0.99946  0.99937
## Pos Pred Value        0.99595   0.9919  0.99502  0.99425
## Neg Pred Value        0.99965   0.9995  0.99902  0.99872
## Prevalence            0.09863   0.1044  0.09752  0.09915
## Detection Rate        0.09832   0.1040  0.09663  0.09800
## Detection Prevalence  0.09872   0.1048  0.09712  0.09857
## Balanced Accuracy     0.99817   0.9973  0.99520  0.99389
```

```
## Confusion matrix for test set
conf_test_dnn<-confusionMatrix(as.factor(pred_test), as.factor(mnist$test$y))
conf_test_dnn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1    2    3    4    5    6    7    8    9
##          0  973    0    2    0    2    2    6    1    5    3
##          1    1 1123    0    0    1    0    2    1    1    2
##          2    0    4 1021    6    3    0    0   10    5    1
##          3    1    1    0  990    0    5    1    2    6    4
##          4    0    0    2    0  966    1    5    0    3   11
##          5    1    1    0    3    0  872    4    0    9    1
##          6    2    1    1    0    4    6  940    0    2    0
##          7    1    1    4    6    0    2    0 1011    4    7
##          8    1    4    2    2    1    2    0    0  934    0
##          9    0    0    0    3    5    2    0    3    5  980
##
## Overall Statistics
##
##                Accuracy : 0.981
##                  95% CI : (0.9781, 0.9836)
##     No Information Rate : 0.1135
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9789
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity            0.9929   0.9894   0.9893   0.9802   0.9837   0.9776
## Specificity            0.9977   0.9991   0.9968   0.9978   0.9976   0.9979
## Pos Pred Value         0.9789   0.9929   0.9724   0.9802   0.9777   0.9787
## Neg Pred Value         0.9992   0.9986   0.9988   0.9978   0.9982   0.9978
## Prevalence             0.0980   0.1135   0.1032   0.1010   0.0982   0.0892
## Detection Rate         0.0973   0.1123   0.1021   0.0990   0.0966   0.0872
## Detection Prevalence   0.0994   0.1131   0.1050   0.1010   0.0988   0.0891
## Balanced Accuracy      0.9953   0.9943   0.9931   0.9890   0.9906   0.9877
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity            0.9812   0.9835   0.9589   0.9713
## Specificity            0.9982   0.9972   0.9987   0.9980
## Pos Pred Value         0.9833   0.9759   0.9873   0.9820
## Neg Pred Value         0.9980   0.9981   0.9956   0.9968
## Prevalence             0.0958   0.1028   0.0974   0.1009
## Detection Rate         0.0940   0.1011   0.0934   0.0980
## Detection Prevalence   0.0956   0.1036   0.0946   0.0998
## Balanced Accuracy      0.9897   0.9903   0.9788   0.9846
```

```
## Accuracies
acc_train_DNN <- mod_dnn %>% evaluate(x_train, y_train)
acc_test_DNN <- mod_dnn %>% evaluate(x_test, y_test)
rbind(acc_train_DNN, acc_test_DNN)
```

```
##               loss       accuracy
## acc_train_DNN 0.03342655 0.99395
## acc_test_DNN  0.1133833  0.981
```

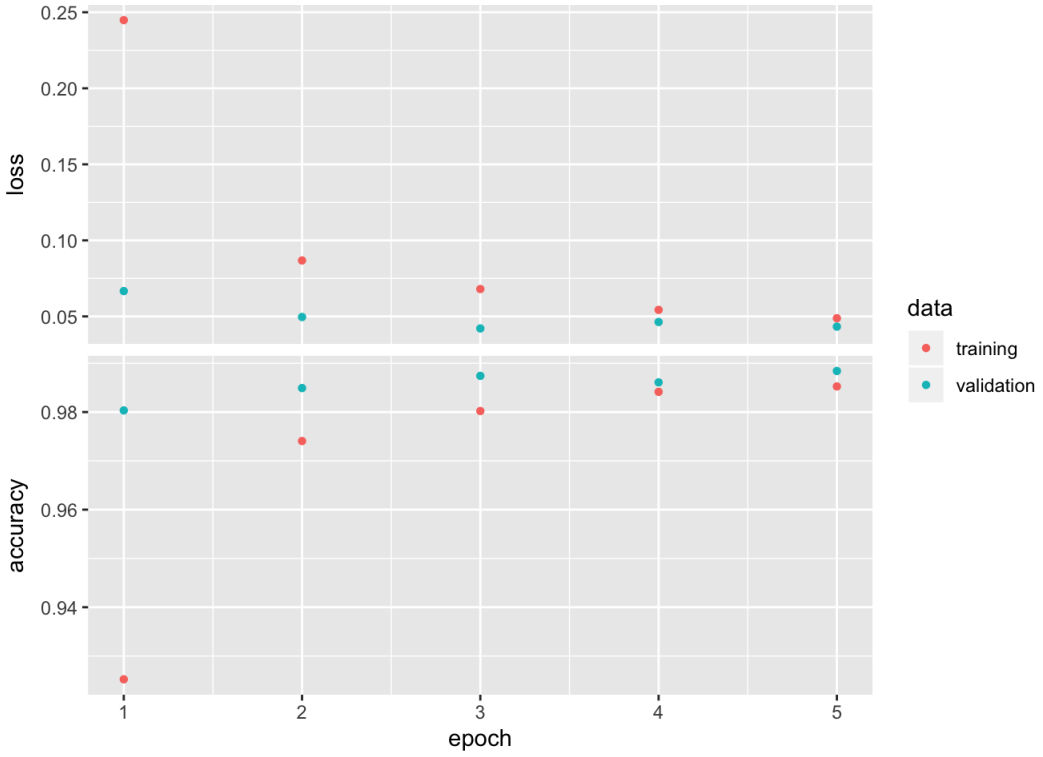
7) Design a CNN and train it on the training set

```
x_train <- mnist$train$x
y_train <- mnist$train$y
x_test <- mnist$test$x
y_test <- mnist$test$y
```

```r
## Reshape
x_train <- array_reshape(x_train, c(nrow(x_train), 28, 28, 1))
x_test <- array_reshape(x_test, c(nrow(x_test), 28, 28, 1))

## Rescale
x_train <- x_train / 255
x_test <- x_test / 255

y_train <- to_categorical(y_train, 10)
y_test <- to_categorical(y_test, 10)

## Design CNN
mod_cnn <- keras_model_sequential()

mod_cnn %>%
  layer_conv_2d(filters = 32, kernel_size = c(3,3), activation = 'relu',
                input_shape = c(28, 28, 1)) %>%
  layer_conv_2d(filters = 64, kernel_size = c(3,3), activation = 'relu') %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_dropout(rate = 0.25) %>%
  layer_flatten() %>%
  layer_dense(units = 128, activation = 'relu') %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 10, activation = 'softmax')

## Compile
mod_cnn %>% compile(
  loss = loss_categorical_crossentropy,
  optimizer = optimizer_rmsprop(),
  metrics = c('accuracy')
)

summary(mod_cnn)
```

```
## Model: "sequential_4"
## ____________________________________________________________________________
## Layer (type)                     Output Shape                  Param #
## ============================================================================
## conv2d (Conv2D)                  (None, 26, 26, 32)            320
## ____________________________________________________________________________
## conv2d_1 (Conv2D)                (None, 24, 24, 64)            18496
## ____________________________________________________________________________
## max_pooling2d (MaxPooling2D)     (None, 12, 12, 64)            0
## ____________________________________________________________________________
## dropout_2 (Dropout)              (None, 12, 12, 64)            0
## ____________________________________________________________________________
## flatten (Flatten)                (None, 9216)                  0
## ____________________________________________________________________________
## dense_9 (Dense)                  (None, 128)                   1179776
## ____________________________________________________________________________
## dropout_3 (Dropout)              (None, 128)                   0
## ____________________________________________________________________________
## dense_10 (Dense)                 (None, 10)                    1290
## ============================================================================
## Total params: 1,199,882
## Trainable params: 1,199,882
## Non-trainable params: 0
## ____________________________________________________________________________
```

```r
## Fit on training
fitted_cnn <- mod_cnn %>% fit(
  x_train, y_train,
  batch_size = 128,
  epochs = 5,
  validation_split = 0.2,
  verbose = 1
)

plot(fitted_cnn)
```



8) Observe the change on performance between a DNN and a CNN, and provide the confusion matrix of the final fitted test set.

```
cnn_pred_train<-mod_cnn %>% predict_classes(x_train)
cnn_pred_test<-mod_cnn %>% predict_classes(x_test)

conf_train_cnn<-confusionMatrix(as.factor(cnn_pred_train), as.factor(mnist$train$y))
conf_test_cnn<-confusionMatrix(as.factor(cnn_pred_test), as.factor(mnist$test$y))

conf_test_cnn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1    2    3    4    5    6    7    8    9
##          0  970    0    1    0    0    1    5    0    1    2
##          1    1 1131    1    0    0    0    2    2    1    2
##          2    1    3 1022    4    0    1    0    7    2    1
##          3    0    0    0 1001    0    4    0    2    0    0
##          4    2    0    3    0  978    0    4    0    0    3
##          5    0    0    0    2    0  883    4    0    0    3
##          6    2    1    0    0    0    2  942    0    0    0
##          7    1    0    4    2    0    0    0 1014    2    2
##          8    2    0    1    1    1    1    1    1  967    2
##          9    1    0    0    0    3    0    0    2    1  994
##
## Overall Statistics
##
##                Accuracy : 0.9902
##                  95% CI : (0.9881, 0.992)
##     No Information Rate : 0.1135
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9891
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity            0.9898   0.9965   0.9903   0.9911   0.9959   0.9899
## Specificity            0.9989   0.9990   0.9979   0.9993   0.9987   0.9990
## Pos Pred Value         0.9898   0.9921   0.9817   0.9940   0.9879   0.9899
## Neg Pred Value         0.9989   0.9995   0.9989   0.9990   0.9996   0.9990
## Prevalence             0.0980   0.1135   0.1032   0.1010   0.0982   0.0892
## Detection Rate         0.0970   0.1131   0.1022   0.1001   0.0978   0.0883
## Detection Prevalence   0.0980   0.1140   0.1041   0.1007   0.0990   0.0892
## Balanced Accuracy      0.9943   0.9977   0.9941   0.9952   0.9973   0.9945
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity            0.9833   0.9864   0.9928   0.9851
## Specificity            0.9994   0.9988   0.9989   0.9992
## Pos Pred Value         0.9947   0.9893   0.9898   0.9930
## Neg Pred Value         0.9982   0.9984   0.9992   0.9983
## Prevalence             0.0958   0.1028   0.0974   0.1009
## Detection Rate         0.0942   0.1014   0.0967   0.0994
## Detection Prevalence   0.0947   0.1025   0.0977   0.1001
## Balanced Accuracy      0.9914   0.9926   0.9959   0.9922
```

9) Compute the performance using an appropriate metric.

```
DNN_train<-acc_train_DNN
DNN_test<-acc_test_DNN
CNN_train<-conf_train_cnn$overall[1]
CNN_test<-conf_test_cnn$overall[1]
cbind(DNN_train,DNN_test,CNN_train,CNN_test)
```

```
##          DNN_train  DNN_test  CNN_train CNN_test
## loss     0.03342655 0.1133833 0.9924667 0.9902
## accuracy 0.99395    0.981     0.9924667 0.9902
```

10) Plot a few examples of misclassified observations in the test set. Which pairs get often misclassified ?

```
set.seed(1)
misclass = sample(which(cnn_pred_test!=mnist$test$y),20)
par(mfrow = c(2,2))

for (i in misclass){
  plot(as.raster(mnist$test$x[i,,], max = 255))
  text(2.5, 2.5, cnn_pred_test[i], cex = 2, col = "light blue")
}
```

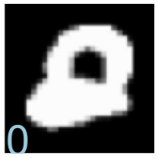
7

8

8

2

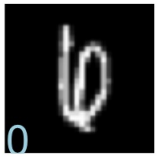
6


0

5

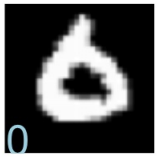
6

2


0

9

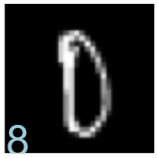
3


0

3

4

8



Accuracy is better on the CNN for the test set, and better on the DNN for the training one. The two most misclassified numbers with a CNN are the 6’s, misclassified as 0, and the 7’s, misclassified as 2.